```
{
    public class CityBreaksContext :
    ➥ IdentityDbContext<CityBreaksUser>                    ❷
    {
        public CityBreaksContext(DbContextOptions options) :
        ➥ base(options)
        {
            ...
```

❶ Add the relevant using directive to bring Identity into scope.

❷ Derive the CityBreaksContext from IdentityDbContext<TUser> instead of DbContext, specifying the CityBreaksUser as the type parameter.

You need to ensure you call `base.OnModelCreating` in your overridden `OnModelCreating` method; otherwise, model configuration for `IdentityDbContext` will not take place, resulting in errors when you attempt to create a migration. This is shown in the bold line in the following listing.

Listing 9.11 Calling `base.OnModelCreating` in the overridden `OnModelCreating`

```
protected override void OnModelCreating(ModelBuilder builder)
{
    builder
        .ApplyConfiguration(new CityConfiguration())
        .ApplyConfiguration(new CountryConfiguration())
        .ApplyConfiguration(new PropertyConfiguration());
    base.OnModelCreating(builder);
}
```

CONFIGURING IDENTITY SERVICES

You will need to configure the application to include Identity services and use the `CityBreaksContext` as a store. Replace the existing call to `AddAuthentication` with the following call to `AddDefaultIdentity<TUser>` in Program.cs, passing in your `CityBreaksUser`.

Listing 9.12 Adding Identity services to the application